

Digital Logic and Design
Smart Plant Monitoring System

Presented By:

Sufiyan Aman

Yousuf Aijaz

Bushra Sadaf

Mubashir Khan



Spring Semester
April, 2024

Contents:

1.Introduction

2.User Flow Diagram

2.1: Current Diagram

2.2: Proposed Diagram

3.Input Block

3.1 CJMCU 6002 UV Sensor Input

3.2. Soil Moisture Sensor Input

3.3. Water Valve

3.4. Initialization in Arduino

3.5. Reading Analog Input

3.6. Code snippets

4. Output Block

4.1. Outputs

4.2. Code snippets

5.Control Block

5.1. States

5.2. State Diagram

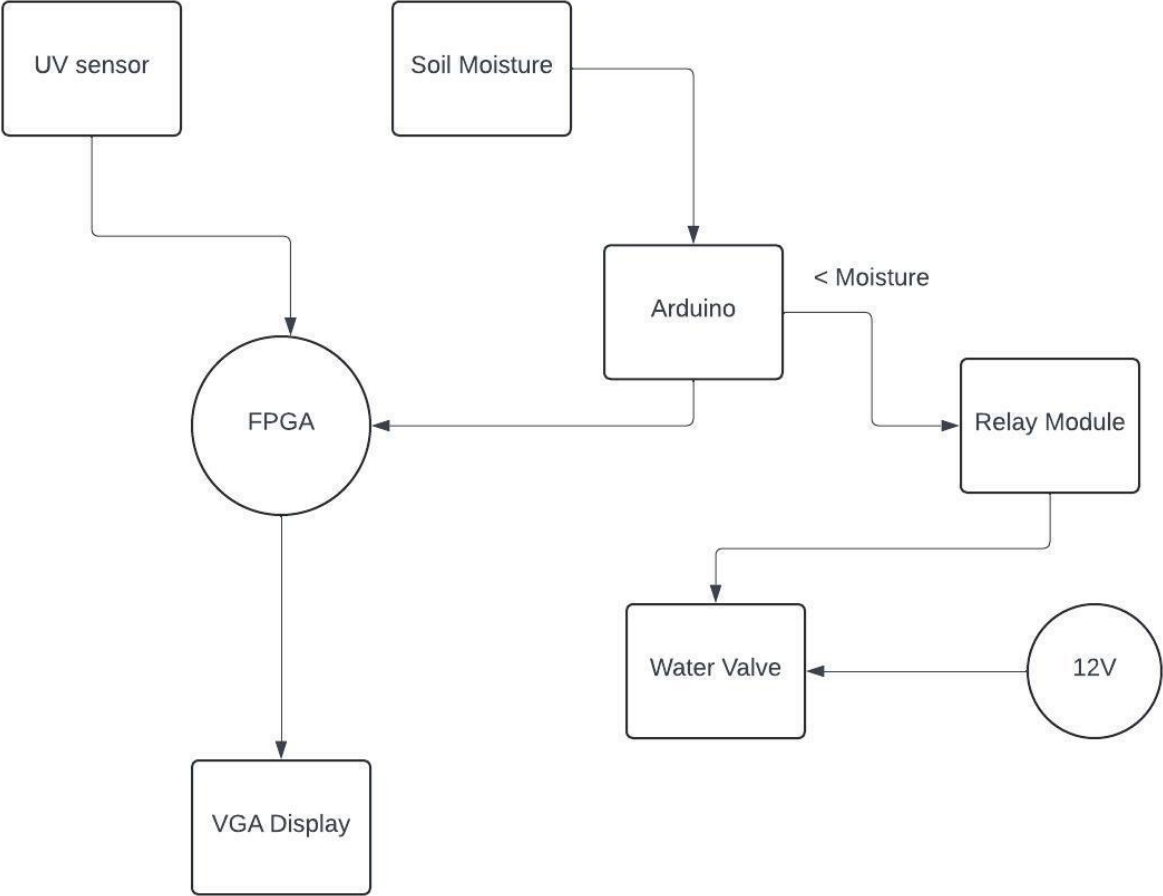
1.Introduction

Our project aims to develop a Smart Plant Monitoring System that utilizes various sensors to monitor the health and environmental conditions of potted plants. This system will provide real-time data on soil moisture, light exposure, and temperature, sending alerts to the user's phone when the plant requires attention, such as watering or adjusting its exposure to sunlight. The target user group for this project includes plant enthusiasts, individuals with busy schedules, or those looking for a convenient way to ensure optimal conditions for their indoor plants.

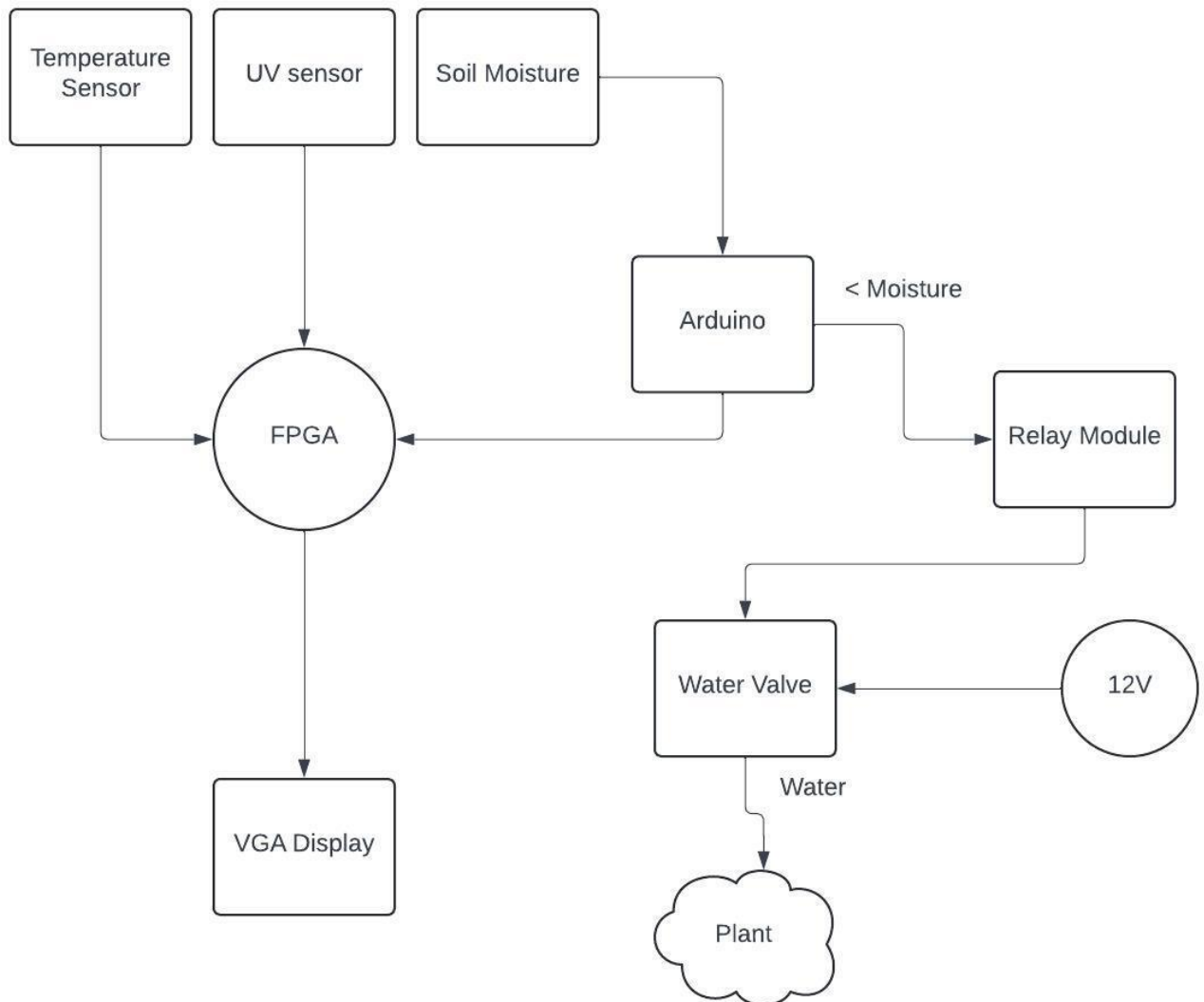
The input module is responsible for linking the sensors (soil moisture, UV, temperature) and the water valve to the FPGA basys 3 board. The output display is responsible for displaying alerts to the user whenever the soil, temperature, or water level rises beyond the defined values.

2. User Flow Diagram:

Current Diagram:



Proposed Diagram:



3. Input Block

3.1 CJMCU 6002 UV Sensor Input:



- **IO Ports:** On the BASYS3 board, the soil moisture sensor is connected using the following pin configuration:

Pmod JA
JA1: J1
JA2: L2
JA3: J2
JA4: G2
JA7: H1
JA8: K2
JA9: H2
JA10: G3

JA1, JA2 are used.

1, Analog Input Pin (ADC_SDA):

The UV sensor provides an analog output voltage representing the UV intensity. In the FPGA circuit, this analog signal is connected to an analog-to-digital converter (ADC), which converts the analog voltage to a digital value. The FPGA reads the digital UV index value from the ADC_SDA pin.

2, Initialization:

In the initialization phase, the FPGA initializes pins and registers.

The uv_index output pin is defined to output the UV index value.

The adc_sda pin, connected to the ADC output of the UV sensor, is defined as an input.

3, Reading Analog Input and Output Control:

In the FPGA module, the UV index value is read from the ADC_SDA pin.

The FPGA processes the digital UV index value obtained from the ADC.

4, Initialization in Setup:

In the setup phase of the FPGA module, L1, L2 pins are initialized.

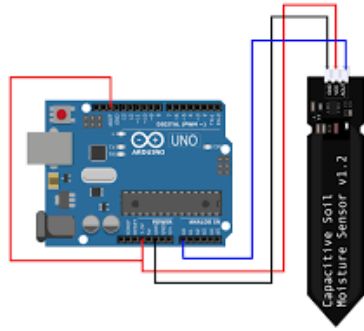
3.2. Analog Input Pin (A0): **Sensor:**

- This pin is connected to an analog sensor, which provides continuous voltage levels.
- The Arduino reads analog values from this pin using the analogRead() function.
- **IO Ports:** On the BASYS3 board, the soil moisture sensor is connected using the following pin configuration:

Pmod JB	
JB1: A14	
JB2: A16	
JB3: B15	
JB4: B16	
JB7: A15	
JB8: A17	
JB9: C15	
JB10: C16	

The soil moisture sensor gives an 8 bit input which is delivered to the above mentioned pins of the fpga.

- An array outputPins() is defined, containing digital output pins (2, 3, 4, 5, 6, 7, 8, 9).
- These pins are used to control the outputs of the water valve and sensors.



Proposed circuit for capacitive soil moisture sensor^

3.3. Valve Relay Pin (12):

- This pin is connected to a relay that controls a water valve.
- **IO Port:** On the arduino, the water valve is defined by pin 12.
- The state of this pin determines whether the valve is open or closed.
- On the BASYS3 board, you would connect this pin to the control input of a relay module.

3.4. Initialization in Setup() Function:

- In the setup() function, the pins are initialized as either INPUT or OUTPUT using pinMode() function calls.
- For the analog input pin, it would be initialized as an INPUT.
- For the output pins array and the relay pin, they would be initialized as OUTPUTs.

3.5. Reading Analog Input and Output Control:

- Inside the loop() function, the Arduino reads the analog sensor value using analogRead() and maps it to a range between 0 and 99.
- The mapped value is then converted into an 8-bit binary number, and each bit is written to one of the output pins to control external devices.
- Additionally, based on the mapped value, the Arduino opens or closes the water valve using the relay pin.

sketch_apr17a.ino

```
1  const int analogInputPin = A0; // Analog input pin
2  const int outputPins[] = {2, 3, 4, 5, 6, 7, 8, 9}; // Output pins array
3  const int valveRelayPin = 12; // Relay pin connected to the water valve
4
5  void setup() {
6      // Initialize each output pin as an output
7      for (int i = 0; i < 8; i++) {
8          pinMode(outputPins[i], OUTPUT);
9      }
10     pinMode(valveRelayPin, OUTPUT); // Initialize the valve relay pin as an output
11     Serial.begin(9600); // Start serial communication at 9600 baud
12 }
13
14 void loop() {
15     // Read the analog input value
16     int sensorValue = analogRead(analogInputPin);
17
18     // Map the sensor value from 99 to 0
19     int mappedValue = map(sensorValue, 0, 1023, 99, 0);
20
21     // Convert the mapped value to an 8-bit binary number
22     for (int i = 0; i < 8; i++) {
23         // Extract each bit of the mappedValue and write it to the corresponding output pin
24         int bitValue = bitRead(mappedValue, i);
25         digitalWrite(outputPins[i], bitValue);
26     }
27
28     // Open the water valve if the mapped value is below 20
29     if (mappedValue < 20) {
30         digitalWrite(valveRelayPin, HIGH); // Open the valve
31     } else {
32         digitalWrite(valveRelayPin, LOW); // Close the valve
33     }
34
35     // Print the analog input, mapped output, and binary output
36     Serial.print("Analog Input: ");
37     Serial.println(sensorValue);
38     Serial.print("Mapped Output: ");
39     Serial.println(mappedValue);
40     Serial.print("Binary Output: ");
41     for (int i = 7; i >= 0; i--) { // Print bits from most significant to least significant
42         Serial.print(bitRead(mappedValue, i));
43     }
44     Serial.println(); // New line after printing all bits
45
46     // Add a small delay before the next reading
47     delay(1000);
48 }
49
```

UV Sensor:

```
1  `timescale 1ns / 1ps
2  module cjmcu_6002_uv_sensor(
3      input clk,
4      input reset,
5      input adc_sda,          // SDA signal from ADC
6      input adc_scl,          // SCL signal from ADC
7      output reg [7:0] uv_index // UV index output
8  );
9
10     // State declarations
11     reg [2:0] state;
12     localparam IDLE = 3'b000, READ_MSB = 3'b001, READ_LSB = 3'b010, OUTPUT = 3'b011;
13
14     reg [3:0] msb;
15     reg [3:0] lsb;
16     reg [1:0] count;
17
18     always @(posedge clk or posedge reset) begin
19         if (reset)
20             state <= IDLE;
21         else
22             case (state)
23                 IDLE: begin
24                     if (count == 2)...
25                 end
26                 READ_MSB: begin
27                     if (count == 2) begin
28                         state <= READ_MSB;
29                         count <= 0;
30                     end
31                 end
32                 READ_LSB: begin
33                     if (count == 2) begin
34                         state <= READ_LSB;
35                         count <= 0;
36                     end
37                 end
```

3. Output Block:

3.1. VGA Display

-The VGA controller is being used to display the sensor data on the screen. The pixel generator module receives the following inputs:

```

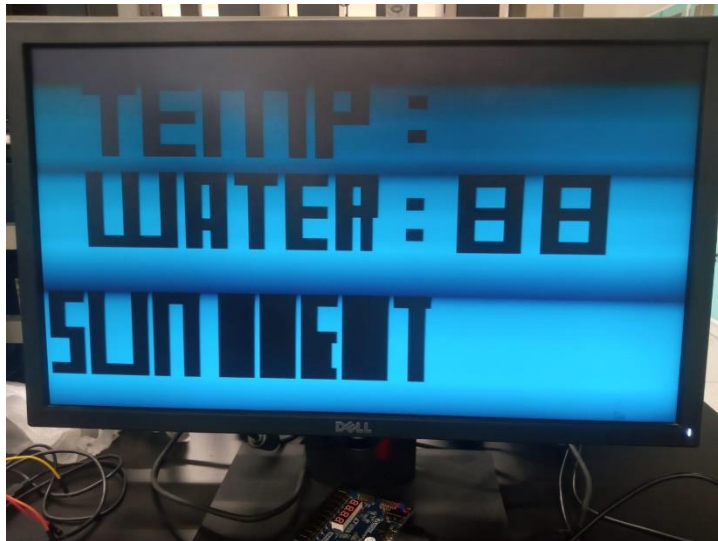
module pixel_gen(
    input clk_d,
    input [9:0] pixel_x,
    input [9:0] pixel_y,
    input video_on,

    input [3:0] B,
    input [3:0] C,
    output reg [3:0] red,
    output reg [3:0] blue,
    output reg [3:0] green
);

```

- The pixel values are generated based on the sensor readings.
- TEMP**: displays the data shown by the temperature sensor
- WATER**: Display the water level
- SUNLIGHT**: display soil moisture sensor reading

Current Display:



```

1  `timescale 1ns / 1ps
2
3  module pixel_gen(
4      input clk_d,
5      input [9:0] pixel_x,
6      input [9:0] pixel_y,
7      input video_on,
8
9      input [3:0] B,
10     input [3:0] C,
11     output reg [3:0] red,
12     output reg [3:0] blue,
13     output reg [3:0] green
14 );
15     always @(posedge clk_d) begin
16     if (
17         //TEMP:
18         //T
19         (pixel_x==50 && pixel_x<=101 && pixel_y>=40 && pixel_y<=60) ||
20         (pixel_x==68 && pixel_x<=83 && pixel_y>=55 && pixel_y<=139) ||
21         //E
22         (pixel_x==110 && pixel_x<=125 && pixel_y>=40 && pixel_y<=139) ||
23         (pixel_x==125 && pixel_x<=160 && pixel_y>=40 && pixel_y<=55) ||
24         (pixel_x==125 && pixel_x<=160 && pixel_y>=82 && pixel_y<=97) ||
25         (pixel_x==125 && pixel_x<=160 && pixel_y>=124 && pixel_y<=139) ||
26         //M
27         (pixel_x==170 && pixel_x<=269 && pixel_y>=40 && pixel_y<=55) ||
28         (pixel_x==170 && pixel_x<=185 && pixel_y>=40 && pixel_y<=139) ||
29         (pixel_x==212 && pixel_x<=227 && pixel_y>=40 && pixel_y<=139) ||
30         (pixel_x==254 && pixel_x<=269 && pixel_y>=40 && pixel_y<=139) ||
31         //P
32         (pixel_x==280 && pixel_x<=295 && pixel_y>=40 && pixel_y<=139) ||
33         (pixel_x==280 && pixel_x<=330 && pixel_y>=40 && pixel_y<=55) ||
34         (pixel_x==280 && pixel_x<=330 && pixel_y>=82 && pixel_y<=97) ||
35         (pixel_x==315 && pixel_x<=330 && pixel_y>=40 && pixel_y<=97) ||
36         //colon

```

4. Control Block

5.1: States:

State 0: Rest

State 1: Sensors are ON

State 2: Soil Moisture Sensor checks the current moisture value

State 3: If moisture<20, enter next state (State 4). Else, moisture>20, and no alert is raised.

State 4: Water Valve is ON and water is sent to the plant.

